

DESIGN DOCUMENT

Cybernara Production Database Schema Specification

Authoritative PostgreSQL/Supabase-compatible schema for the complete Cybernara private-cloud GRC and privacy platform.

Document owner	Swapnil Khedkar
Classification	Cybernara confidential doc
Version	v1.0 - Production Target State
Date	2 July 2026
Deployment	Private cloud - mid-market and enterprise
Deliverable	Logical and physical schema, tables, relationships, constraints, RLS, indexes, partitioning, lifecycle, migrations and verification

CYBERNARA CONFIDENTIAL - Do not distribute without written authorization.

1. Status and Intended Use

This document is the production target-state schema specification derived from the Master PRD, Engineering Requirements, Final Architecture, current schema review, and gap remediation. It defines the complete relational contract and mandatory physical-design controls. Executable migrations must conform to this specification and pass the acceptance suite before production deployment.

Decision	Standard
Database	PostgreSQL 16 or later, Supabase-compatible where Supabase is used.
Authoritative state	PostgreSQL; object bytes in S3-compatible vault. Search, vectors, cache and warehouse are derived projections.
Tenancy	Standard framework catalog is deployment/global. Customer execution and custom overlays are tenant-owned with deny-by-default RLS.
Identifiers	UUIDv7 preferred for new records; stable external/business keys have explicit unique constraints.
Time	timestamptz in UTC; effective-dated relationships use half-open ranges where practical.
Mutation	Optimistic row_version for aggregates; append-only history for answers, approvals, consent, audit and evidence versions.
Extensions	Required: pgcrypto. Optional after review: pgvector, btree_gist, citext.

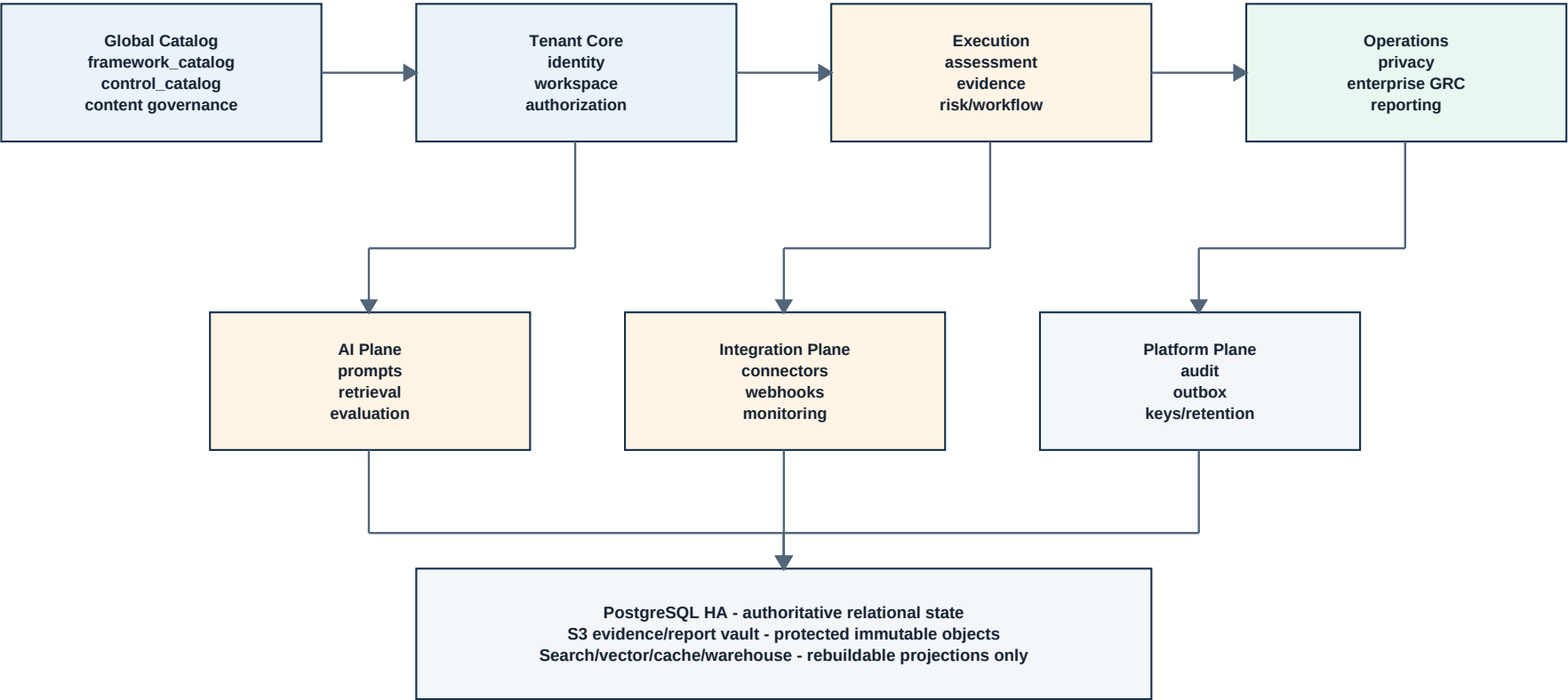
2. Cross-Cutting Column Contract

id uuid PK, tenant_id uuid FK where tenant-owned, created_at timestamptz, created_by uuid, updated_at timestamptz, row_version bigint, classification text, metadata jsonb

- Global catalog tables omit tenant_id and use owner_scope only where customer overlays are supported.
- Mutable aggregate tables include row_version and updated_at trigger; immutable/version tables reject UPDATE/DELETE through trigger and privileges.
- All business enums use lookup tables or CHECK constraints with migration-controlled evolution; free-text status values are prohibited.
- JSONB is allowed for typed payloads, snapshots and extension metadata, not for relationships that require joins, FKs, uniqueness or effective dating.

3. Ownership Architecture

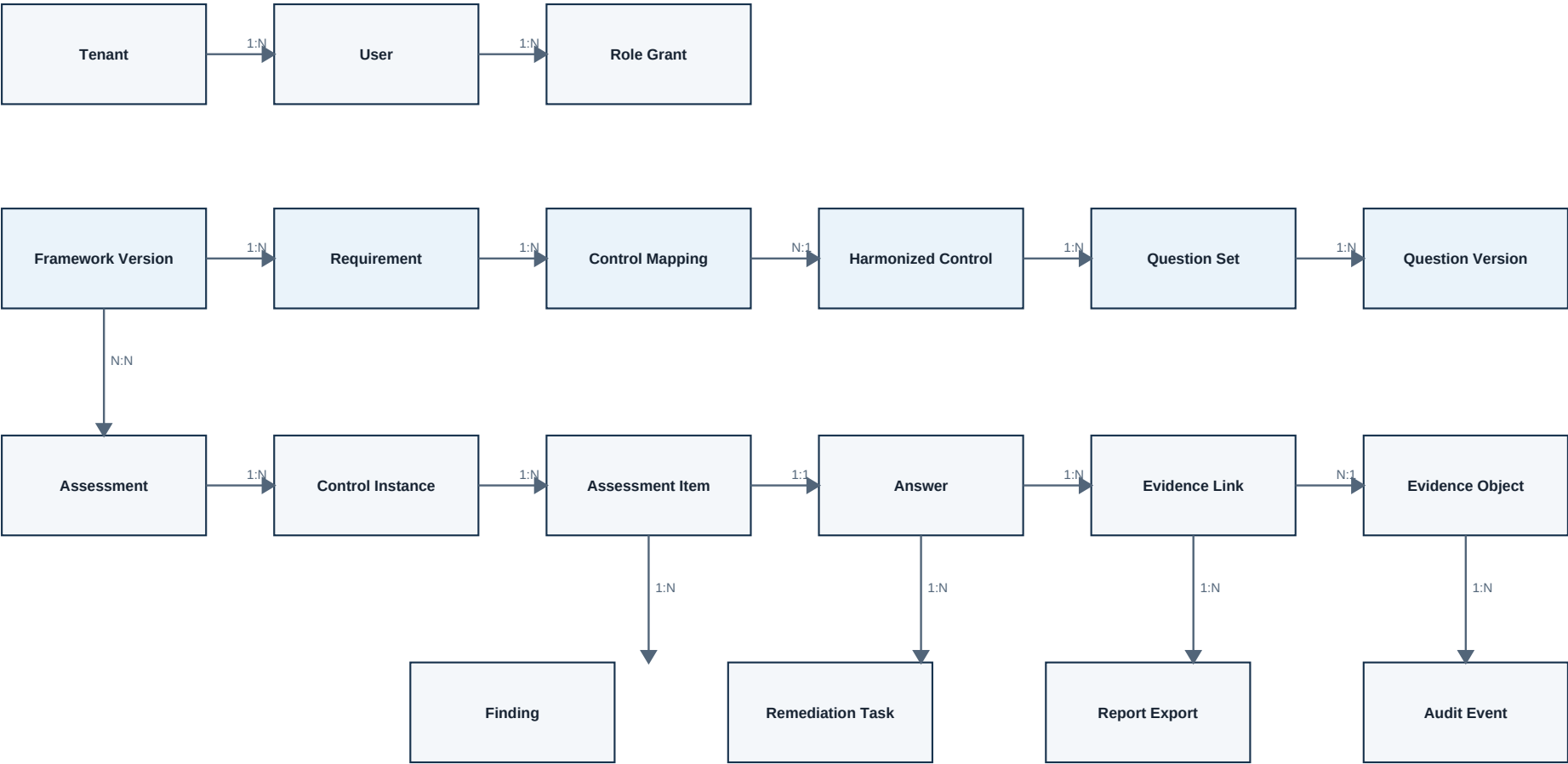
Production Data Architecture and Ownership Boundaries



Global content is versioned and signed. Tenant execution references pinned versions. All tenant access is deny-by-default and policy tested.

4. Core Assessment ERD

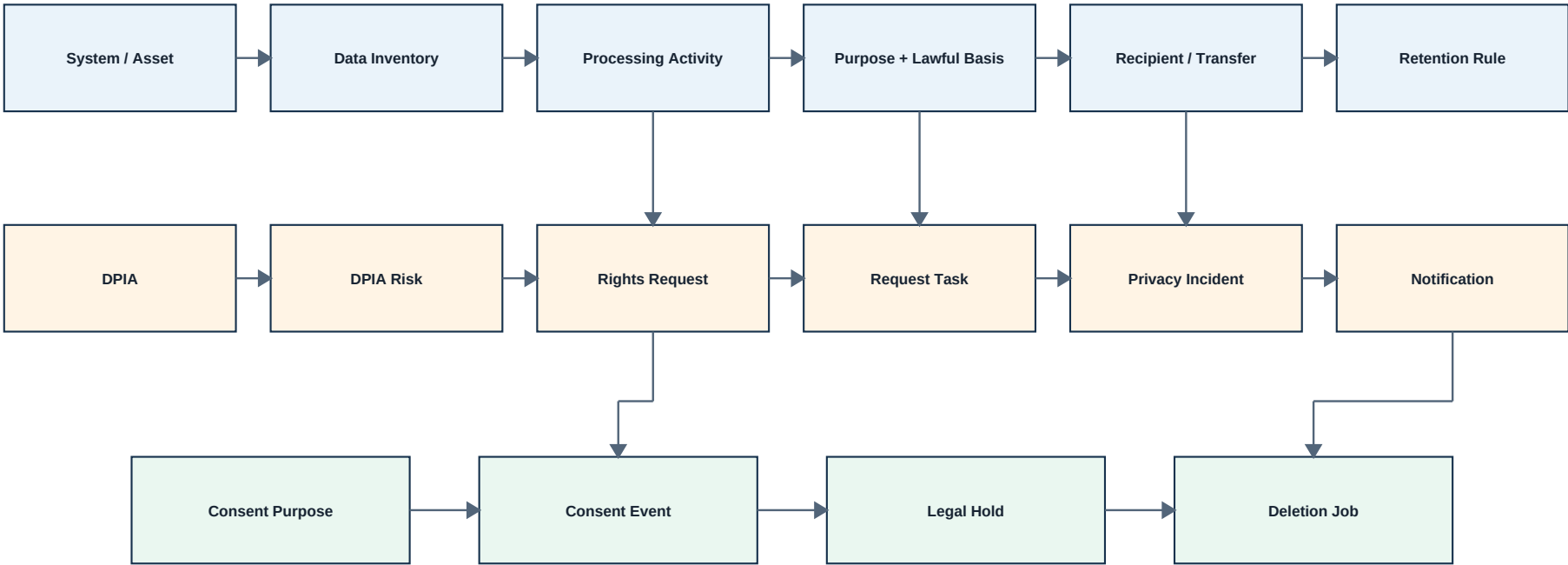
Core Logical ERD



Every mutable business record carries tenant_id, version, created_by, created_at, updated_at, and classification where applicable. Audit events are append-only.

5. Privacy ERD

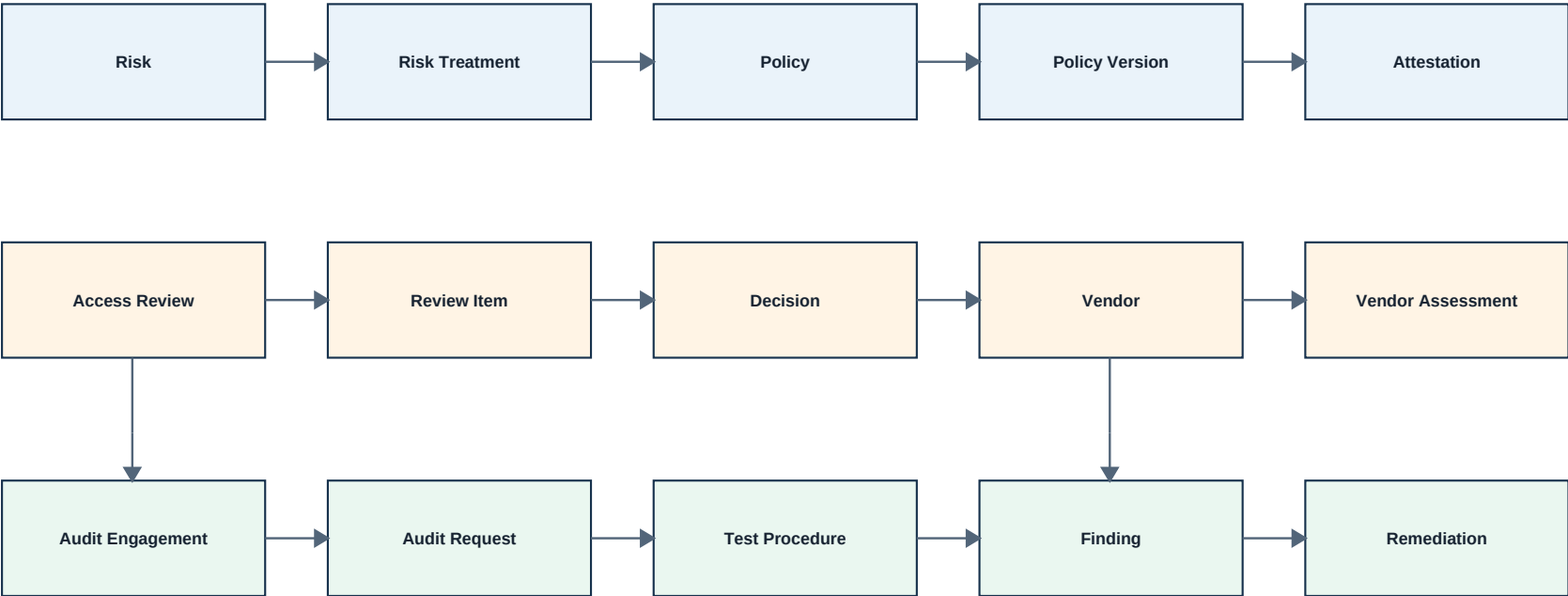
Privacy Operations Logical ERD



Join tables replace UUID arrays: processing_inventory_links, processing_purposes, processing_recipients, processing_transfers, and processing_retention_links.

6. Enterprise GRC ERD

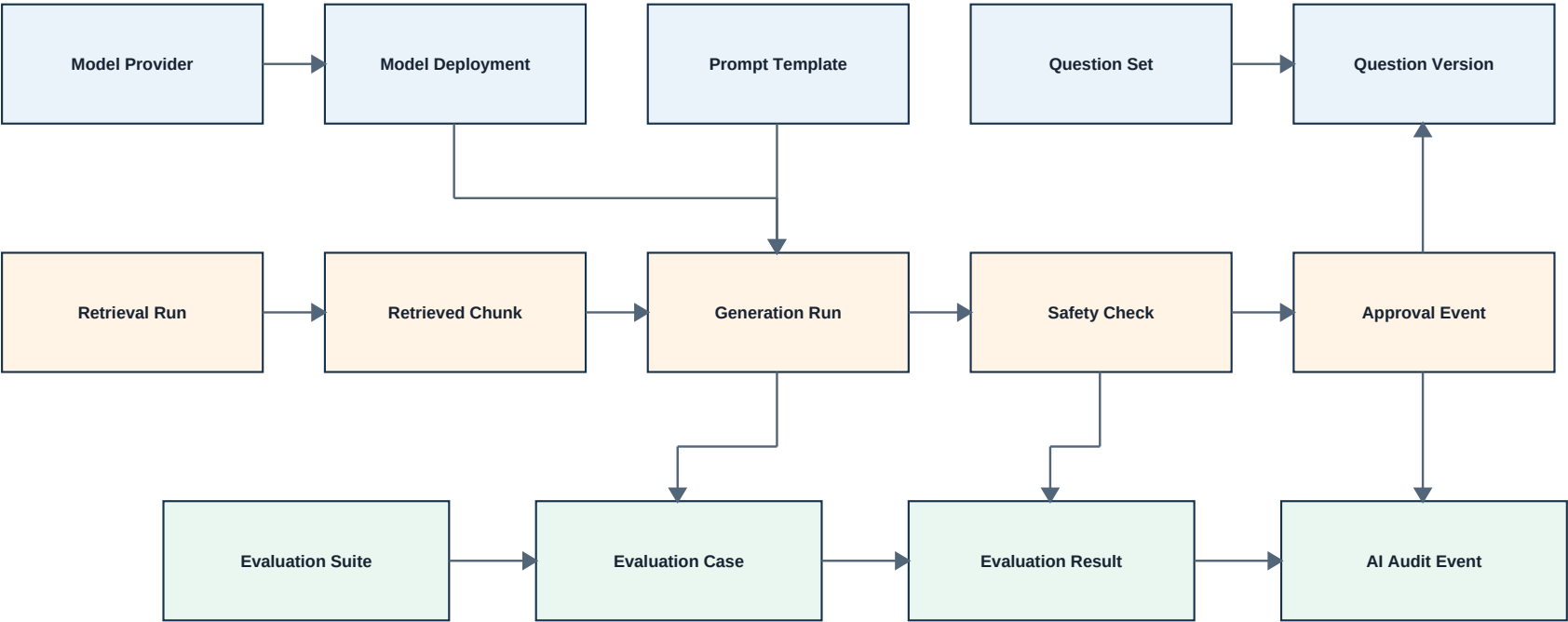
Enterprise GRC Logical ERD



Risks, controls, vendors, policies, audits, findings and evidence connect through typed associative tables with effective dates and approval history.

7. AI Provenance ERD

AI, Provenance, and Evaluation Data Model



Reproducibility key = framework_version + mapping_version + question_version + prompt_version + model_deployment + retrieval_chunk_ids + generation parameters.

8. Identity, Tenant, and Workspace

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
tenants	Customer/deployment tenant	name, slug, status, region, retention_policy_id, current_key_id	unique(slug); status check	status; region
tenant_settings	Versioned tenant configuration	tenant_id, setting_key, value_json, version, effective_at	unique(tenant_id, setting_key, version)	tenant_id, setting_key
users	Human principal	tenant_id, external_subject, email, display_name, status, last_login_at	unique(tenant_id,email); unique(tenant_id,external_subject)	tenant_id,status
groups	Identity/organizational group	tenant_id, external_id, name, source	unique(tenant_id,source,external_id)	tenant_id,name
group_memberships	User-group membership	tenant_id, group_id FK, user_id FK, effective_from, effective_to	unique active membership	group_id; user_id
roles	Role definition	tenant_id nullable, role_key, name, system_role	unique(scope,role_key)	tenant_id,role_key
permissions	Permission catalog	permission_key, resource_type, action	unique(permission_key)	resource_type,action
role_permissions	Role permission assignment	role_id FK, permission_id FK, condition_json	unique(role_id,permission_id)	role_id
role_grants	Principal role grant	tenant_id, user_id/group_id, role_id, workspace_id, granted_by, expires_at	exactly one principal; unique active grant	tenant_id,user_id,workspace_id
sessions	Server session	tenant_id, user_id, token_hash, device_id, expires_at, revoked_at	token_hash unique	user_id,expires_at
service_accounts	Machine principal	tenant_id, name, status, credential_ref, expires_at	unique(tenant_id,name)	tenant_id,status
workspaces	Business unit / operating workspace	tenant_id, parent_id FK, name, workspace_key, status	unique(tenant_id,workspace_key); no cycles	tenant_id,parent_id
workspace_memberships	Workspace participation	workspace_id, user_id/group_id, role_id, effective dates	unique active membership	workspace_id,user_id
delegations	Temporary delegated authority	workspace_id, principal_user_id, delegated_by, scope_json, starts_at, ends_at	ends_at > starts_at	workspace_id,principal_user_id

9. Framework Catalog and Harmonization

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
source_packages	Immutable source import	owner_scope, tenant_id nullable, source_type, filename, uri, sha256, received_at	sha256 unique per owner scope	sha256,status
frameworks	Stable framework identity	framework_key, name, authority, jurisdiction, category	unique(framework_key)	jurisdiction,category
framework_versions	Published framework release	framework_id, version, effective_from/to, source_package_id, status, signature	unique(framework_id,version); nonoverlap active periods	framework_id,status,effective_from
requirements	Canonical obligation	framework_version_id, reference, title, text, requirement_type, source_locator, checksum	unique(framework_version_id,reference)	framework_version_id,reference
requirement_citations	Additional source citation	requirement_id, citation_type, locator, quoted_text_hash	unique(requirement_id,locator)	requirement_id
control_sets	Versioned harmonized library	control_set_key, version, status, source_hash, signature	unique(control_set_key,version)	status
controls	Harmonized control	control_set_id, control_code, domain, title, objective, guidance, evidence_expectation	unique(control_set_id,control_code)	domain,control_code
control_subcontrols	Atomic sub-control	control_id, subcontrol_code, objective, test_guidance	unique(control_id,subcontrol_code)	control_id
mapping_versions	Publishable mapping release	framework_version_id, control_set_id, version, status, approved_by, approved_at	unique(framework_version_id,control_set_id,version)	status
control_mappings	Requirement-control mapping	mapping_version_id, requirement_id FK, control_id FK, coverage, confidence, rationale	unique(mapping_version_id,requirement_id,control_id); coverage check	requirement_id,control_id
mapping_reviews	Four-eyes review	control_mapping_id, reviewer_id, decision, reason, reviewed_at	author != reviewer enforced by policy	control_mapping_id,decision
mapping_conflicts	Detected contradiction/ambiguity	mapping_version_id, requirement_ids, conflict_type, resolution, status	status check	mapping_version_id,status
content_rejections	Import diagnostics	source_package_id, sheet, row_number, source_key, reason_code, raw_hash	unique(source_package_id,sheet,row_number,reason_code)	source_package_id,reason_code
tenant_catalog_subscriptions	Tenant pins/overlays	tenant_id, framework_version_id, control_set_id, mapping_version_id, status	unique active tenant/framework	tenant_id,status

10. Assessment Execution

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
assessment_scopes	Approved structured scope	tenant_id, workspace_id, name, period_start/end, scope_json, approved_by, approved_at	period_end >= period_start	tenant_id,workspace_id
assessments	Assessment aggregate	tenant_id, scope_id, name, status, owner_id, due_at, row_version	status transition policy	tenant_id,status,owner_id
assessment_frameworks	Pinned framework selection	assessment_id, framework_version_id, mapping_version_id	unique(assessment_id,framework_version_id)	assessment_id
assessment_snapshots	Immutable reproducibility root	assessment_id, snapshot_type, sequence, content_hash, created_at	unique(assessment_id,sequence); immutable	assessment_id,sequence
requirement_instances	Pinned mapped or unique obligation	assessment_id, requirement_id, applicability_status, coverage_status, owner_id, status	unique(assessment_id,requirement_id)	assessment_id,status
control_instances	Assessment control execution	assessment_id, control_id, owner_id, applicability_status, status, score, maturity	unique(assessment_id,control_id)	assessment_id,status,owner_id
applicability_decisions	Applicability history	control_instance_id, requirement_id nullable, decision, rationale, decided_by, approved_by	decision check; approver separation	control_instance_id
question_sets	Question collection	control_id, question_set_key, status, source_type	unique(control_id,question_set_key)	control_id,status
question_versions	Immutable approved question payload	question_set_id, version, payload_json, source_generation_id, approved_by, approved_at, checksum	unique(question_set_id,version); immutable	question_set_id,status
assessment_items	Instantiated question	control_instance_id, question_version_id, sequence, required, status	unique(control_instance_id,question_version_id)	control_instance_id,status
answer_revisions	Append-only response revision	assessment_item_id, revision, response_json, submitted_by, submitted_at, supersedes_id	unique(assessment_item_id,revision)	assessment_item_id,revision
review_decisions	Reviewer conclusion	assessment_item_id, answer_revision_id, reviewer_id, decision, rationale, decided_at	reviewer != submitter	assessment_item_id,decision
test_procedures	Versioned control test method	tenant_id nullable, control_id, procedure_key, version, method, expected_result, status	unique scope/control/key/version	control_id,status
control_test_results	Manual/automated test result	control_instance_id, test_procedure_id, run_id, population, sample_json, result, tested_by	result check	control_instance_id,result
assessment_signoffs	Section/final sign-off	assessment_id, scope_type, scope_id, signer_id, decision, signed_at	unique active signoff scope	assessment_id,decision

11. Evidence and Assurance

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
evidence_objects	Logical evidence object	tenant_id, title, owner_id, classification, source_type, state, retention_until	state check	tenant_id,state,owner_id
evidence_versions	Immutable object version	evidence_id, version, object_uri, sha256, size_bytes, mime_type, observed_at, period_start/end	unique(evidence_id,version); sha256 check	evidence_id,sha256
evidence_links	Typed evidence relationship	evidence_version_id, target_type, target_id, purpose, scope_match, period_match	unique(evidence_version_id,target_type,target_id,purpose)	target_type,target_id
evidence_requests	Evidence campaign request	assessment_id, control_instance_id, requested_from, due_at, status, instructions	status check	assessment_id,status,due_at
evidence_reviews	Sufficiency review	evidence_version_id, reviewer_id, decision, rationale, reviewed_at	reviewer separation	evidence_version_id,decision
evidence_samples	Sample definition and selected items	test_result_id, population_ref, method, sample_size, sample_json, seed	sample_size >= 0	test_result_id
malware_scan_results	Upload security result	evidence_version_id, engine, signature_version, status, details_hash, scanned_at	one final result per version/engine	evidence_version_id,status
evidence_expiry_events	Freshness lifecycle	evidence_id, previous_state, new_state, reason, occurred_at	append-only	evidence_id,occurred_at
evidence_custody_events	Chain-of-custody history	evidence_version_id, event_type, actor_id, location_ref, event_hash, occurred_at	append-only	evidence_version_id,occurred_at
automated_tests	Control test definition	tenant_id nullable, control_id, connector_type, query_template, schedule, severity	versioned definition	control_id,connector_type
automated_test_runs	Test execution	automated_test_id, connector_id, started/finished, status, result_json, source_watermark	idempotency unique	automated_test_id,status

12. Risk, Findings, and Workflow

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
risk_models	Versioned scoring methodology	tenant_id nullable, model_key, version, scales_json, formula, thresholds, status	unique scope/model/version	status
risks	Enterprise risk	tenant_id, workspace_id, risk_key, title, category, inherent_score, residual_score, owner_id, status	unique(tenant_id,risk_key); score checks	tenant_id,status,owner_id
risk_links	Risk relation to domain object	risk_id, target_type, target_id, relationship	unique(risk_id,target_type,target_id,relationship)	target_type,target_id
risk_treatments	Treatment plan	risk_id, strategy, plan, owner_id, due_at, status	strategy/status checks	risk_id,status
risk_acceptances	Formal acceptance	risk_id, finding_id nullable, rationale, approver_id, approved_at, expires_at, compensating_controls	expires_at > approved_at	risk_id,expires_at
risk_acceptance_reviews	Periodic review	risk_acceptance_id, reviewer_id, decision, reason, reviewed_at	append-only	risk_acceptance_id
findings	Identified deficiency	tenant_id, assessment_item_id FK nullable, test_result_id FK nullable, title, severity, status	at least one source; severity check	tenant_id,status,severity
remediation_tasks	Corrective action	finding_id, treatment_id nullable, owner_id, status, priority, due_at, verified_at	status transition policy	owner_id,status,due_at
task_dependencies	Task dependency DAG	task_id, depends_on_task_id	no self edge; cycle check service	task_id
workflow_comments	Domain comment	target_type, target_id, author_id, body, visibility	body nonempty	target_type,target_id
approval_requests	Generic approval workflow	target_type, target_id, policy_key, requested_by, status, current_step	status check	target_type,target_id,status
approval_steps	Ordered approval step	approval_request_id, sequence, approver_type/id, decision, decided_at	unique(request,sequence)	approval_request_id
sla_events	SLA clock history	target_type, target_id, event_type, due_at, occurred_at, reason	append-only	target_type,target_id

13. Privacy Operations - Processing and Data Map

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
systems_assets	Systems and repositories	tenant_id, workspace_id, name, asset_type, owner_id, region, criticality	unique tenant/workspace/name	tenant_id,asset_type
data_inventory_records	Data collection/storage record	system_id, data_category_id, location, format, source, steward_id	unique system/category/location	system_id,data_category_id
data_discovery_scans	Discovery/classification run	tenant_id, system_id, connector_id, started/finished, status, classifier_version	idempotency key unique	system_id,status
data_discovery_findings	Discovered data location	scan_id, locator_hash, data_category_id, confidence, sample_prohibited, review_status	unique scan/locator/category	scan_id,review_status
data_categories	Personal/sensitive data taxonomy	tenant_id nullable, category_key, name, sensitivity	unique scope/category_key	sensitivity
data_subject_categories	Subject taxonomy	tenant_id nullable, subject_key, name	unique scope/subject_key	subject_key
processing_activities	RoPA activity	tenant_id, workspace_id, name, controller_processor_role, owner_id, status	unique tenant/workspace/name/version	tenant_id,status
privacy_notices	Stable notice identity	tenant_id, notice_key, audience, owner_id, status	unique tenant/notice_key	tenant_id,status
privacy_notice_versions	Immutable notice version	privacy_notice_id, version, content_uri, sha256, jurisdictions, effective dates, approved_by	unique notice/version; immutable	privacy_notice_id,effective_from
processing_inventory_links	Activity-data inventory join	processing_activity_id, inventory_record_id, role	unique(activity,inventory,role)	processing_activity_id
purposes	Processing purpose	tenant_id nullable, purpose_key, name, description	unique scope/purpose_key	purpose_key
lawful_bases	Jurisdiction lawful basis	framework_version_id, basis_key, name, citation	unique(framework,basis_key)	basis_key
processing_purposes	Activity purpose/basis	processing_activity_id, purpose_id, lawful_basis_id, effective dates	nonoverlap active relation	processing_activity_id
recipients	Recipient/controller/processor	tenant_id, name, recipient_type, country, vendor_id nullable	unique tenant/name/type	tenant_id,type

13. Privacy Operations - Rights, Consent, Incidents, and Retention

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
processing_recipients	Activity recipient join	processing_activity_id, recipient_id, data_categories, purpose	unique typed relation	processing_activity_id
transfers	International transfer	processing_activity_id, from/to country, mechanism, safeguards, status	country/mechanism checks	processing_activity_id,status
dpias	DPIA aggregate	processing_activity_id, trigger_reason, status, owner_id, approved_by, approved_at	status transition	processing_activity_id,status
dpia_risks	DPIA risk	dpia_id, description, likelihood, impact, treatment, residual_score	score checks	dpia_id
rights_requests	Data-subject request	tenant_id, request_type, received_at, due_at, identity_status, status, subject_token	SLA/status checks	tenant_id,status,due_at
rights_request_tasks	Search/decision/fulfillment task	rights_request_id, system_id, owner_id, task_type, status, result_ref	unique request/system/task type	rights_request_id,status
consent_purposes	Versioned consent purpose	tenant_id, purpose_id, notice_version, channel, region, active_from/to	nonoverlap active versions	tenant_id,purpose_id
consent_events	Append-only consent ledger	tenant_id, subject_token, consent_purpose_id, event_type, occurred_at, source, proof_hash	append-only; idempotency key	subject_token,occurred_at
privacy_incidents	Privacy incident	tenant_id, detected_at, severity, status, affected_count, owner_id	severity/status checks	tenant_id,status
incident_assessments	Breach determination	incident_id, jurisdiction, reportable, rationale, assessor_id, decided_at	unique incident/jurisdiction/version	incident_id
incident_notifications	Regulator/subject notification	incident_id, recipient_type, jurisdiction, due_at, sent_at, artifact_id	status derived from timestamps	incident_id,due_at
retention_rules	Retention policy rule	tenant_id nullable, data_category_id, jurisdiction, trigger, duration, disposition	duration positive	data_category_id,jurisdiction
retention_assignments	Rule assignment	retention_rule_id, target_type, target_id, effective dates	nonoverlap active assignment	target_type,target_id
legal_holds	Hold authority	tenant_id, hold_key, reason, issued_by, issued_at, released_at, scope_json	unique tenant/hold_key	tenant_id,released_at
legal_hold_items	Resolved held object	legal_hold_id, target_type, target_id	unique hold/target	target_type,target_id
deletion_jobs	Deletion/erasure execution	tenant_id, trigger, requested_by, status, started/finished	status transition	tenant_id,status
deletion_items	Per-object deletion proof	deletion_job_id, target_type, target_id, disposition, key_destroyed, proof_hash	unique job/target	deletion_job_id,status

14. Enterprise GRC - Policy, Access, Vendor, and Audit

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
policies	Stable policy identity	tenant_id, policy_key, title, owner_id, category, status	unique tenant/policy_key	tenant_id,status
policy_versions	Immutable policy version	policy_id, version, content_uri, sha256, approved_by, effective dates	unique(policy_id,version)	policy_id,effective_from
policy_control_links	Policy-control relation	policy_version_id, control_id, coverage	unique(policy_version,control)	control_id
policy_attestations	Employee attestation	policy_version_id, user_id, decision, attested_at, evidence_hash	unique version/user	user_id
policy_exceptions	Approved policy exception	policy_version_id, subject_type/id, rationale, approver_id, expires_at	expiry required	expires_at
access_reviews	Certification campaign	tenant_id, name, population_source, reviewer_strategy, status, due_at	status transition	tenant_id,status
access_review_items	Reviewable entitlement	access_review_id, principal_ref, resource_ref, entitlement_ref, risk_level	unique campaign/principal/resource/entitlement	access_review_id
access_review_decisions	Certification decision	review_item_id, reviewer_id, decision, rationale, decided_at	one current decision/version	review_item_id,decision
access_remediations	Removal/adjustment result	review_item_id, task_id, external_ticket, status, verified_at	status check	review_item_id,status
vendors	Third party	tenant_id, vendor_key, legal_name, tier, owner_id, status	unique tenant/vendor_key	tenant_id,tier,status
vendor_services	Service/product engagement	vendor_id, name, data_categories, regions, criticality, contract_id	unique vendor/name	vendor_id
vendor_assessments	Due diligence/renewal	vendor_id, assessment_type, period, status, reviewer_id, score	unique vendor/type/period	vendor_id,status
vendor_findings	Vendor deficiency	vendor_assessment_id, severity, title, status, due_at	severity/status checks	vendor_assessment_id
contracts	Contract metadata	tenant_id, vendor_id, contract_key, start/end, renewal_at, document_uri, hash	unique tenant/contract_key	vendor_id,renewal_at

14. Enterprise GRC - Trust, Questionnaires, and Extensions

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
audit_engagements	Audit aggregate	tenant_id, audit_type, scope, period, auditor_org, status	status transition	tenant_id,status
audit_requests	PBC/request list item	audit_engagement_id, control_id nullable, requested_from, due_at, status	unique audit/request key	audit_engagement_id,status
audit_tests	Audit test workpaper	audit_engagement_id, control_instance_id, procedure, sample_ref, conclusion, reviewer_id	versioned workpaper	audit_engagement_id
trust_artifacts	Approved shareable artifact	tenant_id, artifact_type, title, version, object_uri, sha256, approval_id	unique tenant/type/title/version	tenant_id,status
trust_access_requests	Gated artifact access	tenant_id, requester, organization, artifact_scope, nda_status, decision, expires_at	status checks	tenant_id,decision
trust_activity_events	View/download/question activity	tenant_id, access_request_id, event_type, artifact_id, occurred_at	append-only	tenant_id,occurred_at
questionnaire_engagements	Customer/security questionnaire workflow	tenant_id, requester_org, source_uri, due_at, owner_id, status	unique tenant/engagement key	tenant_id,status,due_at
questionnaire_items	Normalized incoming question	engagement_id, sequence, question_text, category, assigned_to, status	unique engagement/sequence	engagement_id,status
questionnaire_responses	Versioned drafted/approved response	questionnaire_item_id, revision, response_text, source_refs, generation_run_id, approved_by, approved_at	unique item/revision; approval required to submit	questionnaire_item_id,revision
custom_object_definitions	Extension object schema	tenant_id, object_key, version, status, validation_schema	unique tenant/object/version	tenant_id,status
custom_field_definitions	Extension field	object_definition_id, field_key, data_type, required, validation_json	unique object/field_key	object_definition_id
custom_records	Extension record	object_definition_id, tenant_id, record_key, status	unique definition/record_key	tenant_id,status
custom_values	Typed extension value	record_id, field_definition_id, value_json, search_text	unique record/field	record_id

15. AI Orchestration and Evaluation

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
model_providers	Approved provider	provider_key, name, data_policy, regions, status	unique provider_key	status
model_deployments	Versioned endpoint	provider_id, deployment_key, model_name, version, region, endpoint_ref, risk_tier, status	unique deployment/version/region	status,region
prompt_templates	Prompt identity	tenant_id nullable, prompt_key, use_case, owner_id	unique scope/prompt_key	use_case
prompt_versions	Immutable prompt	prompt_template_id, version, system_text, output_schema, policy_version, checksum, status	unique template/version	status
retrieval_indexes	Approved index release	tenant_id nullable, index_key, version, embedding_model, source_set_hash, status	unique scope/index/version	status
knowledge_chunks	Versioned retrievable chunk	retrieval_index_id, source_type/id/version, content_hash, acl_json, text_uri	unique index/source/hash	source_type,source_id
retrieval_runs	Retrieval execution	tenant_id, query_hash, filters_json, index_id, top_k, started/finished	top_k bounds	tenant_id,started_at
retrieved_chunks	Ranked retrieval result	retrieval_run_id, knowledge_chunk_id, rank, score, acl_decision	unique run/rank and run/chunk	retrieval_run_id
generation_runs	Generation provenance	tenant_id, prompt_version_id, model_deployment_id, retrieval_run_id, parameters, input/output_hash, status, tokens, cost	idempotency/repro key unique	tenant_id,status
safety_checks	Policy/safety result	generation_run_id, check_type, policy_version, result, score, redaction_summary	unique run/check/policy	generation_run_id,result
generation_citations	Output-source citation	generation_run_id, output_path, knowledge_chunk_id, locator, entailment_score	unique run/output_path/chunk	generation_run_id
output_reviews	Human review	generation_run_id, reviewer_id, decision, rationale, reviewed_at	reviewer separation	generation_run_id
evaluation_suites	Evaluation collection	use_case, suite_key, version, status, threshold_policy	unique use_case/suite/version	status
evaluation_cases	Golden/adversarial case	suite_id, case_key, input_fixture_uri, expected_json, classification	unique suite/case_key	suite_id
evaluation_runs	Evaluation execution	suite_id, prompt_version_id, model_deployment_id, index_id, status, started/finished	repro key unique	suite_id,status
evaluation_results	Per-case/metric result	evaluation_run_id, case_id, metric, score, threshold, passed, artifact_uri	unique run/case/metric	evaluation_run_id,passed
ai_publication_events	Approval to activate output	target_type/id, generation_run_id, approved_version_id, approver_id, published_at	one active publish/version	target_type,target_id

16. Integration and Continuous Monitoring

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
connector_definitions	Connector type/version	connector_key, version, manifest, permissions, status	unique connector/version	status
connector_instances	Tenant connector	tenant_id, definition_id, name, secret_ref, scope_json, status	unique tenant/name	tenant_id,status
connector_sync_runs	Sync execution	connector_id, cursor_before/after, started/finished, status, counts, error_class	idempotency key unique	connector_id,status
connector_objects	External object projection	connector_id, external_id, external_version, object_type, payload_hash, observed_at, deleted_at	unique connector/external_id	connector_id,object_type
connector_health_events	Health history	connector_id, status, latency, error_rate, occurred_at	append-only	connector_id,occurred_at
webhook_contracts	Inbound/outbound contract	tenant_id nullable, webhook_key, version, direction, schema_json, signing_policy	unique scope/key/version	direction,status
webhook_endpoints	Tenant endpoint	tenant_id, contract_id, endpoint_ref, secret_ref, status	unique tenant/contract/endpoint	tenant_id,status
webhook_deliveries	Delivery/receipt	endpoint_id, event_id, signature_valid, attempt, status, response_code, next_retry_at	idempotency/replay key unique	endpoint_id,status
assurance_alerts	Drift/test/connector alert	tenant_id, source_type/id, severity, status, owner_id, detected_at	dedupe key unique active	tenant_id,status,severity
notification_templates	Versioned notification	tenant_id nullable, template_key, channel, version, body_uri	unique scope/key/channel/version	channel
notification_deliveries	Notification outcome	tenant_id, template_id, recipient_ref, target_type/id, status, attempts	idempotency key unique	tenant_id,status

17. Reporting, Audit, and Platform

Table	Purpose	Key domain columns	Critical constraints	Primary indexes
report_templates	Versioned report definition	tenant_id nullable, template_key, version, format, renderer_version, checksum, status	unique scope/key/version/format	status
report_snapshots	Frozen report input	assessment_id, snapshot_sequence, payload_uri, payload_hash, schema_version	unique assessment/sequence	assessment_id
report_exports	Export job/result	tenant_id, snapshot_id, template_id, format, status, object_uri, sha256, signature, completed_at	idempotency key unique	tenant_id,status
export_manifests	Artifact manifest	report_export_id, manifest_uri, manifest_hash, signing_key_id, signed_at	unique report_export	report_export_id
report_schedules	Scheduled distribution	tenant_id, template_id, scope_json, cron, recipients, enabled	schedule validation	tenant_id,enabled
metric_definitions	Versioned metric contract	metric_key, version, formula, dimensions, lineage_json	unique metric/version	metric_key
metric_observations	Materialized metric value	tenant_id, metric_definition_id, dimension_hash, observed_at, value	unique metric/dimension/time	tenant_id,observed_at
audit_events	Immutable event	tenant_id, chain_partition, sequence, actor, action, target_type/id, event_json, previous_hash, event_hash, occurred_at	unique partition/sequence; append-only	tenant_id,occurred_at
audit_checkpoints	Signed chain checkpoint	chain_partition, start/end_sequence, root_hash, signature, signed_at	nonoverlap ranges	chain_partition,end_sequence
audit_verifications	Verifier outcome	checkpoint_id, verified_at, result, mismatch_sequence, verifier_version	append-only	checkpoint_id,result
outbox_events	Transactional async intent	tenant_id, aggregate_type/id, aggregate_version, event_type, payload, idempotency_key, status	unique idempotency_key	status,available_at
inbox_receipts	Consumer deduplication	consumer_key, event_id, received_at, processed_at, status	unique consumer/event	status
authorization_decisions	Policy decision record	tenant_id, principal, action, resource, decision, policy_version, reason, trace_id	append-only	tenant_id,trace_id
rate_limit_policies	Versioned quota	tenant_id nullable, policy_key, subject_type, window, limit, burst, status	unique scope/key/version	status
encryption_keys	KMS key inventory	tenant_id nullable, key_purpose, kms_ref, version, status, activated/retired	one active key/purpose/scope	status
retention_policies	Versioned platform retention	tenant_id nullable, policy_key, target_type, duration, disposition, status	unique scope/key/version	status
backup_restore_tests	Recovery evidence	environment, backup_id, started/finished, rpo_seconds, rto_seconds, integrity_passed, report_uri	append-only	environment,finished_at
release_gates	Release assurance	release_id, artifact_digest, sbom_hash, provenance_ref, scan_summary, approved_by, decision	unique release/artifact	release_id,decision
upload_sessions	Secure upload state	tenant_id, object_key, expected_size/hash, status, scan_status, expires_at	unique object_key; state checks	tenant_id,status
projection_checkpoints	Derived-store cursor	projection_key, tenant_id nullable, source_sequence, target_version, status	unique projection/tenant	status
projection_reconciliation	Source/derived comparison	projection_key, scope, source_count/hash, target_count/hash, result, checked_at	append-only	projection_key,result

18. Relationship and Integrity Rules

Rule	Mandatory implementation
Tenant FK safety	Composite tenant-aware relationships use (tenant_id,id) unique keys and composite FKs where cross-tenant mismatch is possible.
Polymorphic links	target_type/target_id is permitted only for audited extension points. A registry trigger validates allowed target types; core relationships use direct FKs.
No orphan soft delete	Soft deletion uses deleted_at/deleted_by only for recoverable tenant records. Published versions, audit and consent events are never soft-deleted.
Version immutability	Published content, prompts, questions, policy versions, evidence versions and report artifacts reject update/delete. Supersession creates a new row.
Effective dating	Active assignments and versions cannot overlap for the same business key; use exclusion constraints where supported.
Approval separation	Database stores author and approver; policy layer and deferred constraint trigger prevent self-approval for protected operations.
Risk acceptance	Acceptance cannot be represented by task status alone. Active acceptance requires unexpired approval and review schedule.
Report immutability	Completed export must reference frozen snapshot, template/renderer version, stored object, SHA-256, signature and manifest.
Catalog pinning	Assessment creation pins framework_version, mapping_version, control_set and question_version; later releases cannot mutate history.

19. RLS and Authorization

Requirement	Implementation
Default posture	REVOKE public access; enable and FORCE RLS on tenant tables; policies default deny.
Tenant context	Set transaction-local app.tenant_id and app.principal_id from verified server session, never from request body.
Policy shape	USING and WITH CHECK both require tenant_id = current tenant. Workspace/resource policy adds authorized membership or privileged permission.
Service roles	Separate migration, background worker, audit writer and break-glass roles. No shared application superuser. BYPASSRLS is tightly restricted and monitored.
Catalog access	Published global catalog is readable; draft catalog requires content-governance permission. Tenant overlays remain tenant-scoped.
Object/search/cache	PostgreSQL authorization decision precedes signed object URL or derived-store query. Projection ACL fields are defense-in-depth only.
Tests	For every table and operation: same-tenant allow, cross-tenant deny, missing-context deny, forged-context deny, service-role boundary and insert/update WITH CHECK.

20. Index, Partition, and Performance Design

Area	Production rule
FK indexes	Every FK receives a supporting btree index unless proven unnecessary. Composite tenant FKs lead with tenant_id.
Operational queues	Partial indexes on status/available_at for outbox, jobs, alerts, tasks, requests and pending scans.
Time-series partitions	Monthly partitions for audit_events, consent_events, authorization_decisions, metric_observations and high-volume webhook/notification delivery.
Assessment scale	Indexes: assessments(tenant_id,status,updated_at), control_instances(assessment_id,status,owner_id), items(control_instance_id,status).
Evidence	evidence_objects(tenant_id,state,owner_id), versions(evidence_id,version desc), links(target_type,target_id). Object URI never used as business key.
Catalog	requirements/framework_version_id/reference), mappings(mapping_version_id,requirement_id/control_id), trigram/full-text only in derived search.
JSONB	GIN only for demonstrably queried typed metadata; promote frequently filtered keys to columns/generated columns.
Query governance	pg_stat_statements, slow-query SLO, plan regression tests, analyze/vacuum monitoring and index-usage review.
Connection safety	PgBouncer transaction pooling where compatible; bounded pool per workload and statement/lock/idle transaction timeouts.

21. Audit Chain and Tamper Evidence

- Allocate sequence under advisory/row lock per chain_partition; unique(chain_partition, sequence) prevents forks.
- event_hash covers canonical event bytes, previous_hash, sequence and partition. Canonical serialization and hash version are stored.
- Periodically sign checkpoint root hashes with KMS/HSM key; archive events and checkpoint manifests to retention-locked storage.
- Verifier independently recomputes chain and signature and writes audit_verifications. Verification failure raises a critical security alert.
- Business transactions write audit intent and outbox atomically; only the dedicated audit writer appends the final chain.

22. Encryption, Retention, and Privacy Lifecycle

Concern	Mandatory control
At rest	Storage encryption plus envelope encryption for high-sensitivity fields/objects. encryption_keys stores references and lifecycle, never raw key material.
In transit	TLS 1.3 preferred; certificate validation and mTLS for service/data connections where supported.
Field secrets	Connector credentials, tokens and subject identity secrets are vault references or application-encrypted values; never plain JSON.
Retention	retention_rules -> assignments -> scheduled deletion_jobs/items. Legal holds resolve to explicit protected objects before deletion.
Erasure	Delete or anonymize relational data where lawful; destroy per-object/tenant data keys for cryptographic erasure; retain minimum audit proof.
Pseudonymization	Rights/consent subject identifiers use tenant-scoped tokens with segregated lookup where re-identification is required.
Backup lifecycle	Backup retention and deletion align with policy; restore procedure reapplies tombstones/deletion journal before service.

23. Transaction and Event Semantics

- One aggregate mutation and its audit/outbox intents commit in the same PostgreSQL transaction.
- Consumers use inbox_receipts for deduplication. At-least-once delivery is assumed; handlers must be idempotent.
- Event envelope includes event_id, tenant_id, aggregate_type/id/version, event_type/schema_version, occurred_at and trace_id.
- Search/vector/analytics projections advance projection_checkpoints and run reconciliation against source counts/hashes.
- Long imports use staging tables, validation reports and atomic publish; never partially publish catalog versions.

24. Migration and Cutover Standard

Stage	Required action
Design	Schema ADR, data classification, threat model, query patterns, volume forecast and rollback strategy.
Expand	Add nullable/new tables, indexes concurrently, triggers and policies without breaking old application versions.
Backfill	Chunk by stable key, checkpoint progress, throttle load and record source/target counts and hashes.
Dual operation	Dual write or change-capture where required; compare reads and reconcile every difference.
Constrain	Validate NOT NULL/FK/check constraints after clean backfill; enable FORCE RLS and production grants.
Cut over	Feature flag reads to target, monitor SLO/errors, preserve immediate rollback window.
Contract	Remove legacy arrays/columns only after retention period, backup and signed migration acceptance.

25. Backup, Recovery, and DR

- Encrypted continuous WAL/PITR with RPO <= 15 minutes; multi-zone HA and object version replication.
- Backups use separate credentials and failure domain; production application role cannot delete backups.
- Quarterly restore into isolated environment verifies FK/constraint health, tenant/RLS tests, object hashes, report manifests and audit chain.
- Measured restore must meet RTO <= 4 hours for the system of record; exceptions require documented business approval.
- Recovery runbook covers PostgreSQL, object vault, secrets/KMS references, projections, connector cursors and queued events.

26. Production Acceptance Checklist

Gate	Pass criteria
Schema completeness	Every Master PRD P0/P1 persistent concept maps to a table/column/constraint or documented external system.
Integrity	No orphan rows; all mandatory FKs validated; array/JSON relationship backfills reconcile to zero differences.
Isolation	Automated RLS and object/search/cache isolation suite passes for all roles and operations.
Reproducibility	Historical assessment, approved question set and stored report verify after framework/model/template upgrades.
Security	Threat model, ASVS-relevant controls, secrets scan, SQL injection tests and independent penetration test have no unresolved critical/high issue.
Performance	Production-volume load/soak tests meet API/job/report SLOs with acceptable plans and lock behavior.
Recovery	PITR/object restore, RPO/RTO, audit verification and projection rebuild pass.
Migration	Rehearsed expand/backfill/dual-run/cutover/rollback completes with signed reconciliation report.
Operations	Dashboards and alerts cover DB saturation, replication/WAL, partitions, slow queries, queues, RLS failures, audit chain and backups.

27. Implementation Note

This specification is intentionally stricter than a table-existence audit. Production approval requires executable DDL and policies, seeded reference data, migration and rollback scripts, automated integrity/RLS/performance/recovery tests, and evidence that the deployed database matches the signed schema release. No document alone can make a database production-ready; this document defines the contract that implementation and assurance must prove.